

Homework 2

CS 181, Fall 2026

Out: Oct. 3

Due: Oct. 12, 11:59 PM

Please upload your solutions on Gradescope. You can use $\text{\LaTeX}$ or a word document to write up your answers, but we prefer you use $\text{\LaTeX}$. You may scan hand-written work or images for parts of solutions **only if** they are extremely clean and legible. Please ensure that your name does not appear anywhere in your handin.

Problem 1: Free End Gap Alignment

In class you will soon be learning about motifs. Wikipedia defines a motif as “a nucleotide or amino-acid sequence pattern that is widespread and has, or is conjectured to have, biological significance”. Combinatorial pattern matching algorithms are often used to find motifs, but in this homework we’ll see how alignment algorithms can be used for the same task.

Let’s design an alignment algorithm to find motifs for us. Global alignment won’t work for the motif problem because it will align our short motif to a large region of the longer string with many gaps. Since we expect our motif to match on a small area, we might guess that local alignment is appropriate. But local alignment will also have trouble because we want to align the *entirety* of our motif, and local alignment may pick out a subsequence of the motif. To sidestep these problems, we introduce a new kind of alignment: *Free End Gap Alignment*. We want to align the entire motif to a subsequence of the longer sequence, with as few gaps in the aligned section as possible. To this end, we will discount any gaps that occur on the ends of a sequence. For example, the optimal **global** alignment of GTAGGCTTAAGGTTA and TAGATA is

```
GTAGGCTTAAGGTTA
-TAG----A---T-A
```

with a score of -3 (assuming our usual scoring scheme of $+1$ for match, -1 for gap and mismatch). An optimal Free End Gap Alignment, however, would be

```
GTAGGCTTAAGGTTA
-TAGA--TA-----
```

with a higher score of 2 . Indeed, this latter alignment better captures the essence of what it means to find a motif in a longer sequence. We formalize the Free End Gap Alignment problem as follows:

FREE END GAP ALIGNMENT

Input: Strings u , v , and a scoring function δ that takes in two characters as arguments.

Output: An alignment of u , v , for which the score is maximal (as defined by δ) and for which end gaps are free (i.e. not penalized) in the shorter sequence.

- a. Design a dynamic programming solution to the Free End Gap Alignment problem. Your solution must include

- a new recurrence relation, if necessary
- a description of how your table is initialized
- how to backtrack through your table, including where to start and where to end

If you wish, you may also include diagrams, pseudocode, mathematical expressions, and plain English text in your answer. However, please do not submit a block of code with no accompanying explanation.

Now to see the algorithm at work. We've implemented Free End Gap Alignment for you already. Enter your CS181 Projects directory and run `git pull`. This will create the `hw2` directory containing the FEGA scripts: `FEGAlignment.sh` and `fegalignment.pyc`. The `pyc` file is a compiled python file that has been compiled within the course Docker container.

The implementation takes in two arguments, a long sequence and a short sequence to align it against (the scoring scheme is the standard scheme mentioned prior). The code will then output *all* optimal alignments. Here's an example of how to run our implementation:

```
> sh FEGAlignment.sh GTAGGCTTAAGGTTA TAGATA
GTAGGCTTAAGGTTA
-TAGA-T-A-----
GTAGGCTTAAGGTTA
-----TA-GAT-A
```

- b. Use our implementation to count the number of DNA fragments that are created by exposing the following sequence (all three lines make up one contiguous sequence, written 5' to 3') to the restriction enzyme *Bam*HI, which has the recognition site *G*/GATCC. (Note, / denotes the cut site of the enzyme). How would your answer change if the polarity of the following sequence was reversed? **Please enter the Docker container before running our implementation. You may have to run the command `chmod 777 FEGAlignment.sh` if the script won't execute.**

```
TCCATTGATGCCACGGCGGATCCTGGAGAGCAGCAGCGACTTGCATACATCAGATCAGAGTAATACTAGC
ATGCGATAAGTCCCTAACTGACTATGGATCCTTCTAGAGTCAACTTCAGGACATATGGTCTCTGGATCCC
GTGGATCCTTCCTAGGAATCAGATTGGATCCTGGTTAACCATCAAACAGGTCTTGAGTCTAAAATTGTGC
```

Problem 2: Affine Gap Alignment

Affine gap penalties provide a more biologically realistic model for sequence alignment by charging differently for gap opening versus gap extension. We will define affine alignment in this problem as we have been in class, using the objective function

$$\alpha(\# \text{ matches}) - \beta(\# \text{ mismatches}) - \gamma(\# \text{ gap clusters}) - \epsilon(\# \text{ single-letter gaps})$$

Consider the sequences:

u = ACGTAG

v = AGTAG

With scoring parameters: $\alpha = +2, \beta = 1, \gamma = 3, \epsilon = 1$.

- The affine gap alignment algorithm uses four matrices: V , E , F , and G . Define what each matrix represents and write the recurrence relations for all three matrices. Be sure to specify the initialization conditions.
- Using the sequences and parameters above, fill in the four matrices V , E , F , and G for positions $(i, j) = (2, 2)$, $(2, 3)$, and $(3, 2)$. Show your calculations step by step.
- Compare the optimal alignment score of the full length sequences using affine gaps versus linear gaps (with gap penalty = -2). Which alignment would be preferred biologically and why?

Problem 3: Homology

Finding conserved patterns across different species is important for evolutionary biology. Consider the following sequences:

A : 5' AGCTTCGAAGTTATCTTGGACGGACTTG 3'
B : 5' AGTTTCCCAGGATATCTTCGAACGACTG 3'
C : 5' AGGCTTCCCATCCTCCTATAAAGGTAGG 3'

We wish to find whether B or C is a homologous protein to A, that is, a protein that originates from some ancestral species. The optimal alignment for A and B is

A: AGCTTCGAAGT-TATCTTGGA-CGGACTTG
B: AGTTTCCCAGGATATCTTCGAACG-ACT-G

with a score of 12. The optimal alignment between A and C is

A: AG-CTTCGAAGT--TATCT-TGGACGGACTTG-
C: AGGCTTCCCA-TCCTC-CTATAAA-GG--TAGG

with a score of 1. After splicing the DNA into cells, you find that the DNA actually transcribes the following protein sequences (using the one-letter amino acid code):

A: SSYLGR
B: SDIFER
C: SSYKGR

A gene is composed of segments of protein-coding DNA called *exons*, which are split by segments of noncoding DNA called *introns*. We can split genes A, B, and C into introns and exons as follows (exons are underlined and all other regions represent introns).

A :5' AGCTTCGAAGTTATCTTGGACGGACTTG 3'
B :5' AGTTTCCCAGGATATCTTCGAACGACTG 3'
C :5' AGGCTTCCCATCCTCCTATAAAGGTAGG 3'

- Notice how similar A and C are as proteins despite being much more dissimilar than A and B as DNA sequences. Give a biological reason for this observation (*hint*: look at the segmentation of the genes into introns and exons!). Why might it be a better idea to align by amino acid sequence rather than DNA?
- There are 20 different amino acids and only 4 different nucleotides (used in DNA). Give a probabilistic reason why aligning by amino acid sequence might be better than by DNA sequence.

Problem 4: Statistical Foundations of Sequence Alignment

The most popular type of substitution matrices are known as BLOSUM matrices. Blocks, or fixed regions in a given set of aligned sequences, are used to create these substitution matrices. We are studying sequence similarities among some breeds of cattle on Sorin's farm and will analyze BLOSUM matrices derived from their genomes.

Given an alphabet, $\Sigma = \{\alpha, \beta, \gamma, \delta\}$, the following block contains the sequences of a single, 3-residue protein from each of nine cattle breeds: angus, belted galloway, brahman, charolais, dexter, gelbvieh, hereford, holstein, limousin.

Angus	α	γ	δ
Belted Galloway	β	γ	α
Brahman	γ	γ	γ
Charolais	β	β	γ
Dexter	β	γ	δ
Gelbvieh	γ	α	δ
Hereford	α	β	β
Holstein	γ	γ	δ
Limousin	β	γ	β

By determining the evolutionary information in the above block, we will develop an improved scoring scheme for sequence alignment.

- What is the total number of residues we have in this block? How many ways can we pick a pair of letters in each column? How many possible ways of picking letters from the above table are there? Assume the paired letters must be in the same column.
- Determine the observed frequencies for each alignment pair.
- For each alignment pair, determine the expected frequency based on the above block, and calculate the log-likelihood score, f , that compares the observed and expected frequencies for each pair.
- Explain the meaning of positive, negative, and zero log-likelihood scores. Given this interpretation, why would using log-likelihood scores as the scoring scheme be desirable for sequence alignment?
- Interpret the final log-likelihood scores in this problem in terms of what they indicate about conservation of letters in this alphabet. Comment on how realistic this scoring scheme would be if we were to use it to align actual genome sequences.
- Bonus:** More formally, we can express the log-likelihood score of aligning two letters a and b as:

$$f_{ab} = 2 \log_2 \frac{p_{ab}}{q_{ab}}$$

where:

- $p_{ab} = \mathbb{P}(a, b|M)$ is the joint probability of observing aligned letters a and b under the alignment model M

- ii. $q_{ab} = \mathbb{P}(a, b|R) = \mathbb{P}(a|R)\mathbb{P}(b|R)$ is the joint probability of observing aligned letters a and b under a random model where letters appear independently.

Recall that local alignment requires the expected score of aligning two letters to be non-positive to ensure that alignments remain local. Therefore, if we want to use BLOSUM matrices for local alignment, we need to ensure that the expected score f_{ab} is not positive.

Prove that the expected score $\mathbb{E}[f_{ab}]$ for a randomly aligned pair of letters is not positive for any alignment model M .

Hint: For any concave function g , $\mathbb{E}[g(X)] \leq g(\mathbb{E}[X])$.

Problem 5: Multiple Sequence Alignment

The following problem will teach you how [CLUSTAL](#), a popular **multiple sequence alignment** (MSA) algorithm, aligns three or more sequences at the same time.

For example:

```
S1  A C G T A
S2  A C - T A
S3  A C G T G
```

CLUSTAL uses a heuristic called progressive multiple sequence alignment, where a **guide tree** determines the order in which sequences and partial alignments (called **profiles**) are merged. For example, the guide tree $((A,B),(C,D))$ means that sequences A and B are aligned first, sequences C and D are aligned next, and then the resulting two profiles are aligned. To align two profiles, CLUSTAL uses a natural extension of the Needleman-Wunsch algorithm. The difference is that the rows and columns of the dynamic-programming table correspond to *alignment columns*, and not individual characters. As for scoring, when two profile columns are aligned, you compute the transition score by summing the scores of all comparisons between a character in the first profile column and a character in the second profile column. This results in the following recurrence relation for aligning two profiles P_A and P_B , where a gap column is a column comprised of only $-$.

$$F(i, j) = \max \begin{cases} F(i-1, j-1) + \text{score}(P_{A,i}, P_{B,j}), \\ F(i-1, j) + \text{score}(P_{A,i}, \text{gap column}), \\ F(i, j-1) + \text{score}(\text{gap column}, P_{B,j}). \end{cases}$$

Consider the following four sequences that we want to align using the guide tree $((S_1, S_2), (S_3, S_4))$:

$S_1 : G \ A \ C \quad S_2 : G \ - \ C \quad S_3 : G \ T \ C \quad S_4 : G \ T \ -$

We will be using the following scoring scheme (note that there is a separate score for a gap aligned to a gap, which wasn't present for pairwise alignment!):

Comparison	Score
Match	+1
Mismatch	-1
Character-gap	-2
Gap-gap	0

Thus, after the first two steps of the algorithm, the two profiles to be aligned are:

$$P_{S_1 S_2} = \begin{matrix} G & A & C \\ G & - & C \end{matrix} \quad P_{S_3 S_4} = \begin{matrix} G & T & C \\ G & T & - \end{matrix}$$

- a. Fill in the dynamic-programming table (with backpointers!) below for a global alignment of profiles $P_{S_1S_2}$ and $P_{S_3S_4}$. Show your work for at least one of the nonzero rows/columns. What is the final multiple sequence alignment?

$P_{S_1S_2} \setminus P_{S_3S_4}$	0	1	2	3
0				
1				
2				
3				

- b. Suppose that a different guide tree were used, such as

$$((S_1, S_3), (S_2, S_4)).$$

Briefly explain why a different guide tree could produce a different final multiple alignment.

- c. Suppose there are k sequences, each with length approximately n . Briefly explain why progressive alignment is faster, and provide at least one situation each where it would be more appropriate to use one or the other.